



Experiential Learning - IVth Semester

SentinelSound

An Edge-AI Acoustic Monitoring System for Real-Time Poaching Detection and Ranger Alerting in Protected Forests

Snehal Reddy Thadigotla(1RV24IS126)

Sumukha Upadhyaya(1RV24IS130)

Introduction

Real-Time Acoustic Anti-Poaching

01

The Poaching Problem

Gunshots in vast reserves are over in seconds — patrols cannot be everywhere.

02

The Detection Gap

By the time a shot is heard or found, poachers are gone.

03

Our Approach

Low-cost acoustic nodes listen 24/7 and classify gunshots with edge + ML inference.

04

Actionable Outcome

A geo-tagged, confidence-scored alert reaches rangers instantly for rapid response.

The Poaching Problem: Illegal hunting is among the gravest threats to endangered wildlife, and vast reserves such as Bandipur are impossible to patrol on foot continuously.

The Detection Gap: Gunshots and chainsaws are over in seconds; by the time rangers hear or discover them, the poachers are already long gone.

Our Approach: EnviEL deploys low-cost acoustic sensor nodes that listen continuously and use on-device plus edge machine learning to recognise gunshots in real time.

The Actionable Outcome: The moment a gunshot is classified, a geo-tagged, confidence-scored alert reaches rangers through a live dashboard, enabling rapid response.

Problem Statement

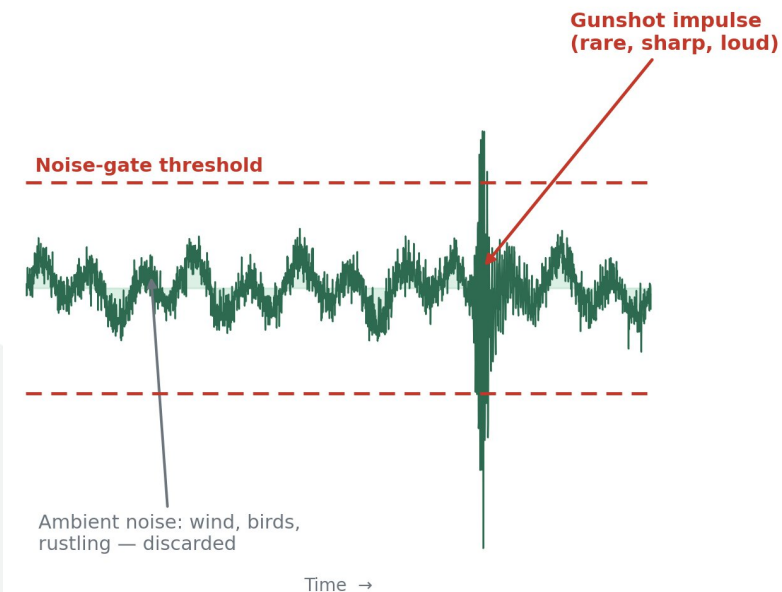
Reactive, Not Proactive: Conventional anti-poaching relies on physical patrols and post-incident investigation, missing the brief window when intervention is still possible.

An Acoustic Needle in a Haystack: A gunshot lasts milliseconds amid hours of wind, birdsong, and rustling, so naive amplitude alarms are buried under constant false positives.

Power & Connectivity Limits: Sensors must run on battery in remote forest with intermittent connectivity, ruling out heavy, cloud-only audio streaming for every second of sound.

Fragmented Tooling: Detection, verification, mapping, and patrol coordination usually live in disconnected tools, slowing the ranger's decision loop in the field.

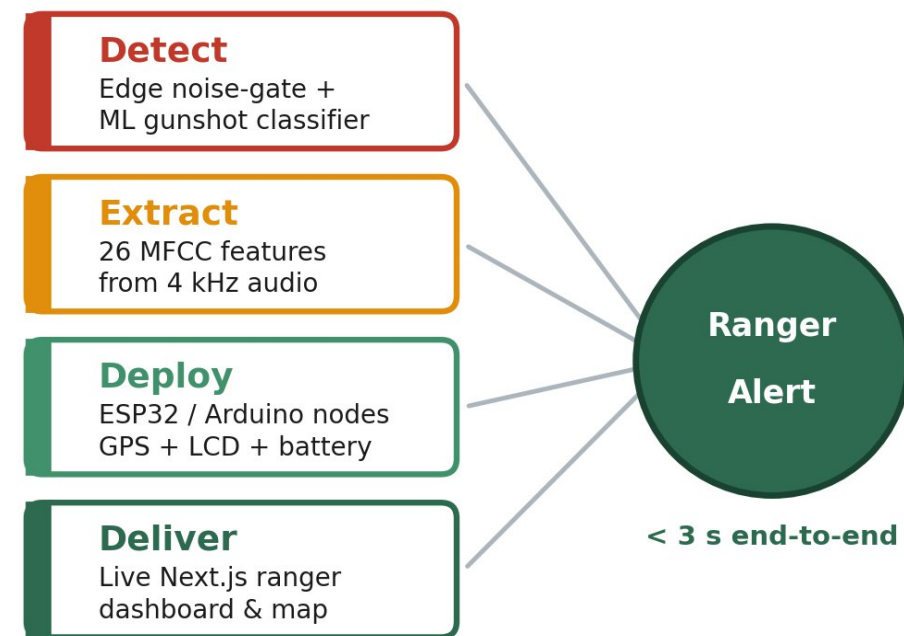
A gunshot is an acoustic needle in a haystack



Primary Objectives

- **Build a Real-Time Gunshot Detector:** Combine an edge noise-gate with a trained ML classifier to flag gunshots from continuous audio stream in real time.
- **Engineer an Acoustic Feature Pipeline:** Convert raw 4 kHz audio into 26 MFCC features (13 means and 13 standard deviations) for robust, lightweight classification.
- **Deploy Geo-Tagged Sensor Nodes:** Run the detector on ESP32 and Arduino Uno nodes with GPS position, a 16x2 LCD status display, and battery telemetry.
- **Deliver a Live Ranger Dashboard:** Surface every detection on a live map with confidence, severity, and a verification workflow in a Next.js web app.

System at a Glance



Secondary Objectives

- **Suppress False Positives:** Apply a two-stage noise gate so only loud impulses ever reach the ML model, saving battery power and on-device compute.
- **Calibrate Confidence & Severity:** Map each detection to a graded severity (critical, high, medium) and a confidence score (0.82 to 0.97 for gunshots) for triage.
- **Add a Voice AI Assistant:** Integrate a Gemini-powered, voice-enabled assistant so rangers can query events and dispatch patrols hands-free in the field.
- **Ensure Resilient Operation:** Support NTP-synced timestamps, heartbeat health checks, and node auto-re-registration after reboot for reliable field uptime.

Literature Review

- **Davis & Mermelstein (1980), "Comparison of Parametric Representations for Monosyllabic Word Recognition" (IEEE TASSP):** Introduced Mel-Frequency Cepstral Coefficients (MFCC), the acoustic feature representation used in this project. Limitation: designed for speech, so less tuned to short transient impulses such as gunshots.
- **Salamon & Bello (2017), "Deep CNNs and Data Augmentation for Environmental Sound Classification" (IEEE Signal Processing Letters):** Showed deep CNNs with augmentation excel at environmental sound classification, including a gunshot class. Limitation: deep models are too heavy for microcontroller-class edge devices.
- **Chacon-Rodriguez et al. (2011), "Evaluation of Gunshot Detection Algorithms" (IEEE Trans. Circuits and Systems I):** Compared acoustic features and classifiers for detecting gunshot signatures in noisy environments. Limitation: tuned to specific weapons and ranges, limiting generalisation.

Literature Review

- **McFee et al. (2015), "librosa: Audio and Music Signal Analysis in Python" (Proc. SciPy):** Provides the MFCC extraction pipeline used by the serial bridge in this project. Limitation: not optimised for real-time feature extraction on embedded hardware.
- **Warden & Situnayake (2019), "TinyML: ML on Ultra-Low-Power Microcontrollers" (O'Reilly):** Established that always-on audio inference can run on milliwatt-class edge devices. Limitation: focuses on TensorFlow Lite Micro rather than classical ML pipelines.
- **Piczak (2015), "ESC: Dataset for Environmental Sound Classification" (ACM Multimedia):** Introduced a benchmark of environmental sounds, including gunshot-like classes, for training and evaluation. Limitation: clean clips differ from noisy real-forest acoustics.

Methodology

Step 1: Acoustic Sensing & Noise Gating

- An Arduino Uno samples the microphone at 4 kHz (8-bit). A pre-detection window tracks peak-to-peak amplitude; only impulses above the noise-gate threshold ($\sim 120/255$) arm a full 1-second capture, so ambient sound is never processed.

Step 2: Feature Extraction

- Each triggered 1-second window (4000 samples) is streamed over serial to a Python bridge, re-centred to the $[-1, 1]$ range, and converted with librosa into 26 MFCC features (13 means and 13 standard deviations).

Step 3: ML Classification

- The 26-feature vector is fed to gunshot_animal_model.pkl, a trained scikit-learn binary classifier that outputs a gunshot / not-gunshot label and a probability, $P(\text{gunshot})$.

Methodology

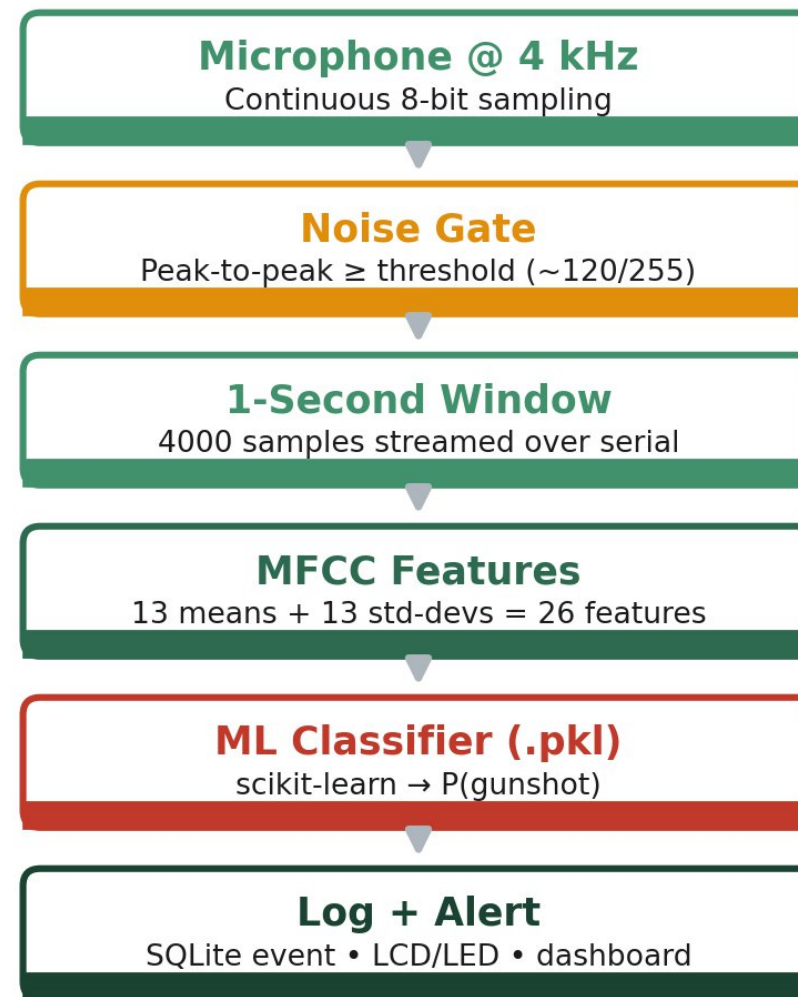
Step 4: Event Logging & Alerting

- Positive detections are written to a SQLite store (poaching_events) with node ID, severity, confidence, and timestamp; a result byte (G/N) returns to the Arduino to update the LCD and flash the alert LED.

Step 5: Visualisation & Response

- A Next.js dashboard renders events on a Leaflet map with Recharts analytics. Key pipeline parameters:
 - **Sample rate: 4 kHz** | **Window: 1 s (4000 samples)**
 - **Features: 26 MFCC** | **Edge latency: < 3 s end-to-end**
- A verification workflow (pending, verified, false-positive) and a Gemini voice assistant let rangers triage events and dispatch patrols.

Detection Pipeline

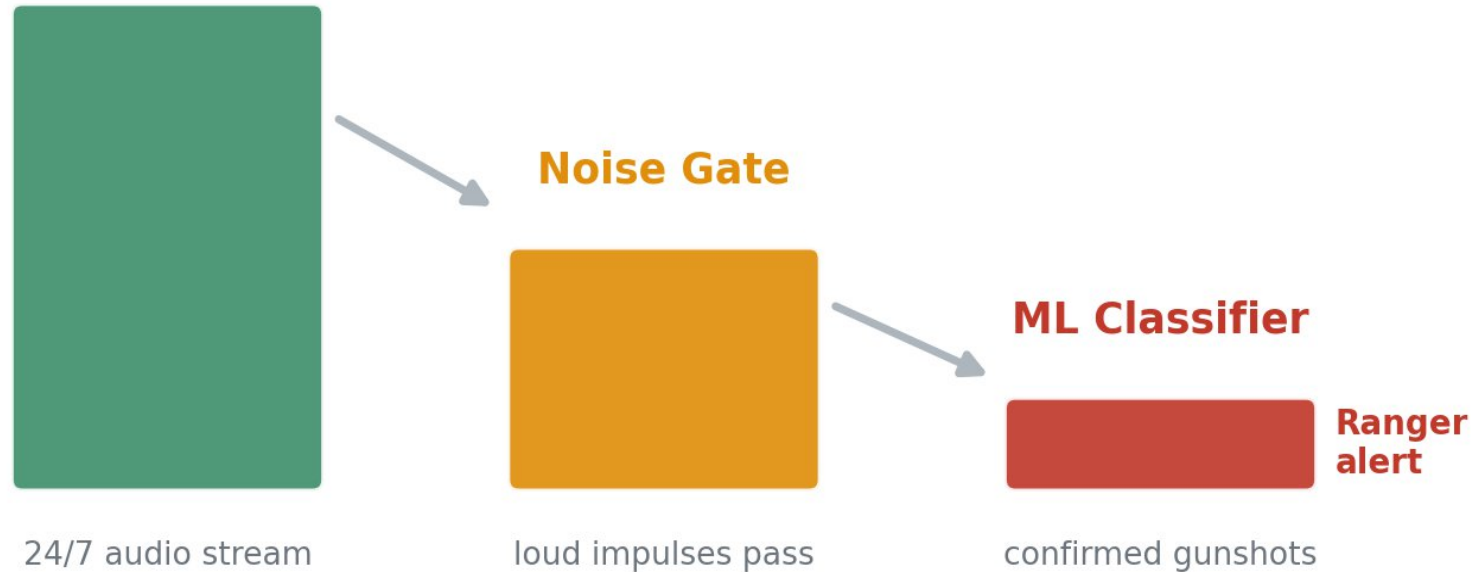


Methodology

Two-Stage Filtering Keeps Alerts Clean

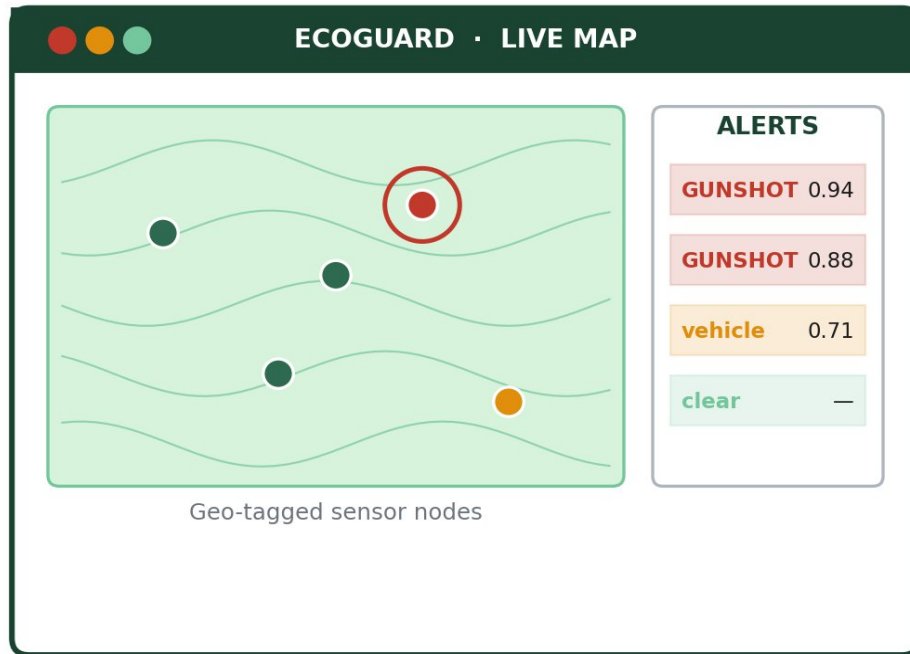
Volume of audio ↓ at each stage

Continuous



Detection latency: < 3 s
Sample rate: 4 kHz
Inference window: 1.0 s
Features: 26 MFCC |
Gunshot confidence:
0.82–0.97

Methodology

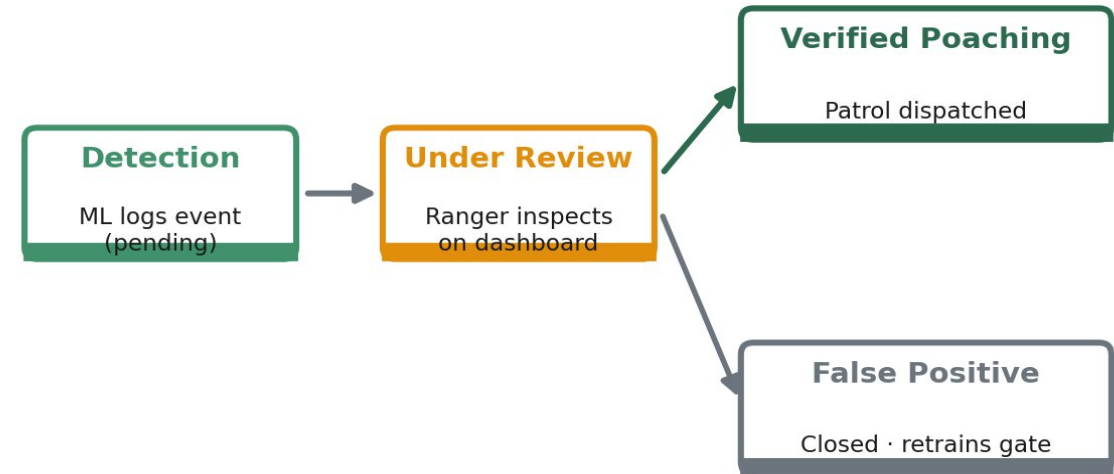


Live Operations Dashboard

Shows:

Geo-tagged detections across sensor nodes, with a live alert feed and confidence scores for rapid ranger response.

Event Verification Workflow



Event verification workflow: each detection moves from pending to verified poaching or false positive, driving patrol dispatch and tuning the noise gate.

Tools and Technologies Used

- **Edge Hardware & Firmware:**
 - ESP32 & Arduino Uno (ATmega328P); MAX9814 / KY-038 sound sensor; 16x2 I2C LCD and status LEDs
 - C++ (Arduino) firmware; 4 kHz ADC sampling; two-stage noise gating on-device
- **Machine Learning & Audio:**
 - Python 3, scikit-learn (binary classifier, gunshot_animal_model.pkl)
 - librosa (MFCC feature extraction); NumPy; joblib (model loading)
 - pySerial (Arduino-to-host serial bridge)
- **Web, Backend & Visualisation:**
 - Next.js 16, React 19, TypeScript; node:sqlite (SQLite); Google Gemini (assistant + transcription)
 - Tailwind CSS, Radix UI; Leaflet & Recharts; Git & GitHub (version control)

References

- Davis, S. B., & Mermelstein, P. (1980). Comparison of Parametric Representations for Monosyllabic Word Recognition in Continuously Spoken Sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(4), 357-366.
- Salamon, J., & Bello, J. P. (2017). Deep Convolutional Neural Networks and Data Augmentation for Environmental Sound Classification. *IEEE Signal Processing Letters*, 24(3), 279-283.
- Chacon-Rodriguez, A., Julian, P., Castro, L., Alvarado, P., & Hernandez, N. (2011). Evaluation of Gunshot Detection Algorithms. *IEEE Transactions on Circuits and Systems I: Regular Papers*, 58(2), 363-373.
- McFee, B., Raffel, C., Liang, D., Ellis, D. P. W., McVicar, M., Battenberg, E., & Nieto, O. (2015). librosa: Audio and Music Signal Analysis in Python. *Proceedings of the 14th Python in Science Conference (SciPy)*, 18-25.



References

- Warden, P., & Situnayake, D. (2019). TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers. O'Reilly Media.
- Piczak, K. J. (2015). ESC: Dataset for Environmental Sound Classification. Proceedings of the 23rd ACM International Conference on Multimedia, 1015-1018.
- Pedregosa, F., Varoquaux, G., Gramfort, A., et al. (2011). Scikit-learn: Machine Learning in Python. Journal of Machine Learning Research, 12, 2825-2830.
- Vercel. (2024). Next.js (Version 16) [Computer software]. <https://nextjs.org>
- EnviEL: SentinelSound Anti-Poaching Acoustic Monitoring System [Source Code]. (2026). GitHub. <https://github.com/Suchi9126/EnviEL>